5DATA005C.1 – Data Engineering-Documentation

Lecturer name – Mrs. Yaalini Balathasan

Student name – S.M.G. Dulwan Senarath

UoW ID – w2120563

IIT ID – 20240258

Tutorial Group - B

Table of Contents

Objectives 2

Data sources 3

Scenario 1: Database for Content Based Image Retrieval (CBIR) system. 4

- Methodology 4
  - Image collection and data sourcing 4
  - Image preprocessing 6
  - Image annotation and metadata generation 13
  - image feature extraction 14
- Database implementation and storage: 17
  - Querying : 20

Scenario 2: Database for sentiment analysis models. 23

- Methodology 23
  - Text collecting and sourcing 23
  - Text preprocessing 23
  - Feature extraction / vectorization 25
  - Sentiment labeling 26
  - Creating files to save in the database 27
- Database implementation and storage 28
  - Querying: 29

Evaluation 30

Conclusion 31

References 32

- Images dataset: 32
- Text dataset: 32

Objectives

The aims of this coursework are to design, implement and document two NoSQL databases that are able to handle unstructured data types, namely images and texts. The first database implements a Content-Based Image Retrieval (CBIR) in which images are stored with associated annotation metadata and visual features that are extracted. This database is meant to assist in effective retrieval of images based on their visual similarity instead of reliance upon textual filenames. The CBIR database shows how complex, rich multimedia data can be managed by NoSQL systems through the extraction of meaningful image features and associating those features with descriptive metadata.

The second database is for sentiment analysis, using textual data. It houses customer reviews, along with outputs from preprocessing, two various vectorizations, and labels of sentiments This will enable analysis of opinions expressed in text data and illustrate how textual information can be represented numerically in structured formats that allow for efficient storage and querying in a NoSQL environment. These two databases together will provide a vision of flexibility and suitability for NoSQL databases in the management, storing, and retrieving of different forms of unstructured data.

Data sources

1. Two different types of datasets were used in this coursework: image data for the CBIR system and textual data for the sentiment analysis system. The image dataset includes a sum of 75 images sourced from a single image-sharing and photography website, which is publicly available. These images were organized into meaningful categories such as,

- Food (12 images)
- Photography (22 images)
- Product (13 images)
- Dubai (14 images)
- Transportation (14 images)

All raw images were stored such prior to pre-processing, as the entire dataset has been placed in cloud storage using an accessible link .

Images cloud folder link = https://drive.google.com/drive/folders/1Ubj3BhhP-yH8Kx6ehAYA_hG180ltCoZp?usp=sharing

2. The text data consists of 50 customer reviews related to the Mazda 3 vehicle. Each review represents a single document and primarily expresses and opinion or sentiment. All customer reviews have been gathered manually from the source and stored in a single raw text file before applying any preprocessing steps were applied. Similar to the image dataset, this text data also been uploaded to a cloud storage folder.

Text cloud file link = https://drive.google.com/file/d/1BI13Auigno4yWERYa-FLMe_fj5frltmh/view?usp=sharing

All data sources have been documented to facilitate traceability, which allows verification of the datasets, if needed.

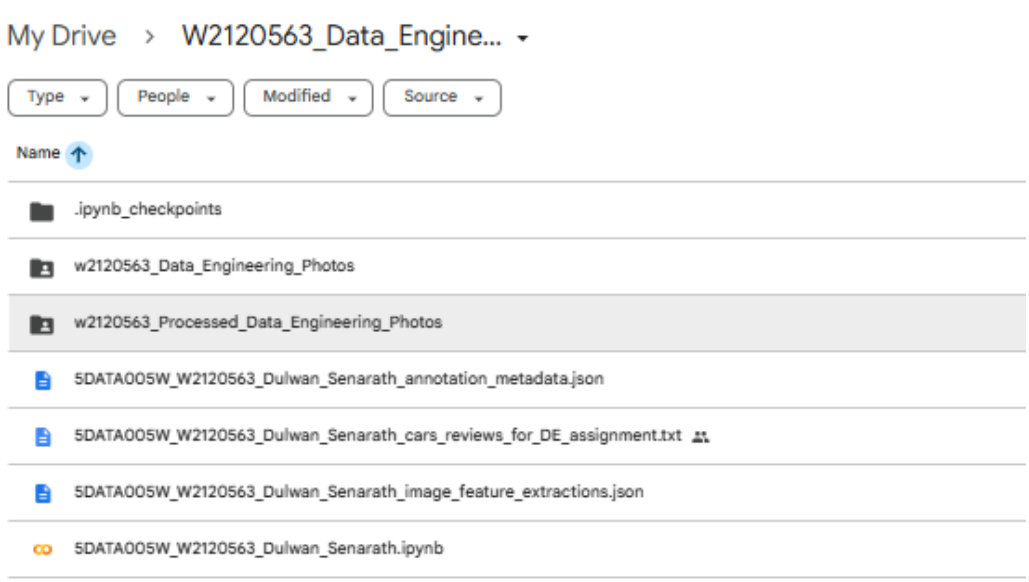
Scenario 1: Database for Content Based Image Retrieval (CBIR) system.

Methodology

Image collection and data sourcing

The dataset was constructed by collecting diverse set of digital images that represent multiple real-world domains, such as cityscapes, food, transportation, products and photography. The images that are sourced are easily accessible through a systematically created directory created in the Google Drive (named 5DATA005W_Data_Engineering_Assesment_2) that enables scalable processing. The main reason behind the selection of a variety images was to ensure that the dataset encompasses a variety of characteristics like color variations, level of details, shapes and scene concepts that can be quarried when placed in a database environment.

After images were loaded (into "W2120563_Data_Engineering_Photos" folder) a for loop was implemented to ensure accessibility, integrity, and correct loading behavior prior to preprocessing. This ensures that any potentially malformed files, which may be of an unsupported type, do not propagate into the subsequent stages of the pipe.



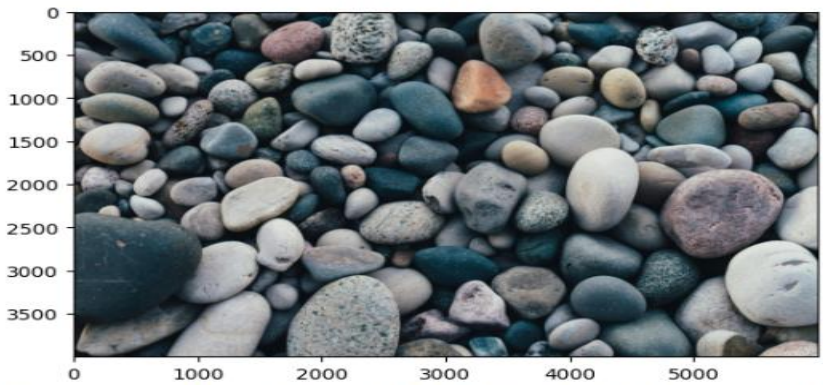
Code:

part a : image collection

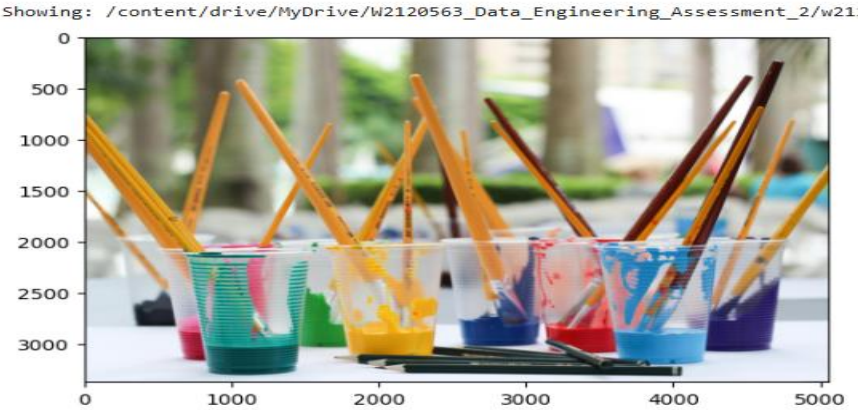
```
folder_path = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Data_Engineerin

image_files = glob.glob(folder_path + "/*.*")

for img_path in image_files:
    try:
        img = Image.open(img_path)

        print("Showing:", img_path)

        plt.imshow(img)
        plt.show()

    except Exception as e:
        print("Skipping:", img_path, "Error:", e)
```

Example Output:

```
Showing: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Data_Engineering_Phot
```




```
Showing: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Data_Engineering_Phot
```




```
Showing: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Data_Engineering_Phot
```

Image preprocessing

All the images underwent pre-processing and the size of the images was standardized. The images were all resized to the 500x500 pixels that make sure consistent spatial dimension across the dataset as requested in the assignment, making the images better for future analysis. The decision eliminates bias caused by different image resolutions, lowers computing overhead, and enhances feature extractions.

Gaussian blurring was applied as a denoising technique; it reduces high-frequency image noise without significantly affecting essential structural information. This step becomes of primary importance in case of texture and edge-based feature extraction, because excessive noise distorts the statistical measures of texture and contour detection. All preprocessed images were saved with systematic filenames into a separate directory to preserve the original data and ensure reproducibility.

After these steps, the pre-processed photos are saved in the directory in a folder named "W2120563_Processed_Data_Engineering_Photos" with each imaged saved in this folder having "_processed" text after the original name it had before.

Code:

part b : image preprocessing

```
from PIL import Image, ImageFilter


raw_dir = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Data_Engineering_Photos"
processed_dir = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos"



os.makedirs(processed_dir, exist_ok=True)


image_paths = glob.glob(raw_dir + "/*.*")

for path in image_paths:
    try:

        img = Image.open(path).convert("RGB")


        resized = img.resize((500, 500))


        denoised = resized.filter(ImageFilter.GaussianBlur(radius=1.5))


        base = os.path.basename(path)
        name, ext = os.path.splitext(base)
        save_name = f"{name}_processed{ext}"


        save_path = os.path.join(processed_dir, save_name)
        denoised.save(save_path)

        print("Saved:", save_path)

    except Exception as e:
        print("Skipping:", path, "| Error:", e)

print("\n✔ All images resized to 500x500 and Gaussian-denoised!")
```

Example output:

```
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/transportat
Saved: /content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos/photography
```

After this step, the processed images are displayed:

Code:

```
folder_path = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Photos"


image_files = glob.glob(folder_path + "/*.*")


for img_path in image_files:
    try:
        img = Image.open(img_path)

        print("Showing:", img_path)

        plt.imshow(img)
        plt.title(os.path.basename(img_path))
        plt.show()

    except Exception as e:
        print("Skipping:", img_path, "| Error:", e)
```

few processed images:

Example processed image 1

Example processed image 2

Example processed image 3

Example processed image 4

Example processed image 5

Image annotation and metadata generation

For the semantic metadata, a manual creation in the form of keywords and a natura-language description was done for every image. This tagging facilitates the assigning of meaning to unstructured visual information to form a NoSQL database where the images can be understood for search, filtering, or interpretation.

Keywords were used as a way to condense information about the most visible objects, actions, or motifs, and the full description supplies a more elaborate narrative. The metadata was expressed in JSON for because this is a lightweight and human-readable markup schema and is natively supported by all NoSQL databases such as MongoDB.

Codes: Creating and labeling text data.

Part C : image annotation

```
metadata={
    "dubai_image_1_processed":{
        "keywords": ["dubai skyline","palm island","luxury villas","blue lagoon"],
        "description": "A wide aerial view overlooking Dubai's skyline with blue waterfront, palm-shaped islands, and rows c
    },
    "dubai_image_2_processed":{
        "keywords": ["dubai marina","sky scrapers","yachts","aerial view"],
        "description": "A bright aerial view of a modern waterfront cityscape featuring tall skyscrapers surrounding a turqu
    },
    "dubai_image_3_processed":{
        "keywords": ["futuristic architecture","night cityscape","Arabic calligraphy","iconic structure"],
        "description": "A nighttime cityscape featuring a striking futuristic building shaped like a torus, covered in glowi
    },
    "dubai_image_4_processed":{
        "keywords": ["traditional architecture","old stone staircase","heritage building","Middle Eastern courtyard"],
        "description": "A rustic outdoor staircase made of stone and mud-brick, featuring curved traditional wall designs."
    },
    "dubai_image_5_processed":{
        "keywords":["Burj Al Arab","Coastline","Beach","Luxury resort"],
        "description": "A wide sandy beach stretches along a calm shoreline with clear blue water having a unique shaped bui
    },
    "dubai_image_6_processed":{
        "keywords": ["Follow-me travel","Urban wanderlust","Futuristic cityscape","Turquoise waterfront"],
        "description": "A traveler stands on a waterfront walkway, reaching forward as if inviting the viewer along, with Du
    },
    "dubai_image_7_processed":{
        "keywords": ["Supercar lineup","Dubai street scene","Urban lifestyle","Modern city streets"],
        "description": "A striking lineup of brightly colored supercars—yellow, red, and green—parked along a modern city st
    },
    "dubai_image_8_processed":{
        "keywords": ["Urban megastructure","Daytime cityscape","Rapid urbanization","toll road"],
        "description": "A striking modern skyline rises against a clear blue sky, showcasing a dense cluster of futuristic s
```

Code to save as JSON file:

```
import json

with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/5DATA005W_W2120563_Dulwan_Senarath_annotation_metac
    json.dump(metadata, f, indent=4)

print("JSON file created successfully!")
```

```
JSON file created successfully!
```

image feature extraction

in order to transform visual content into machine-interpretable data, several feature extractions methods were used.

- Color intensity features: This feature calculates mean pixel intensity values to capture overall brightness and color distribution for every single image, facilitating high-level comparisons among images

- Texture features: Statistical texture parameters (contrast, homogeneity and energy) were computed from the Gray Scale Co-occurrence Matrix (GCLM) to quantify surface morphology and texture uniformity. These properties are highly effective for natural scene, architectural structure and food texture analysis.

- Shape features: The most accurate edge detection and contour analysis methods were utilized to detect object borderlines. On the contours, region shape characteristics which were area, perimeter, centroids, and bounding box were computed to record the spatial and geometric information.

Feature data was structured in JSON format to ensure consistency, extensibility, and compatibility with database storage and analytical queries.

part d : image feature extraction

```python
from skimage.feature import graycomatrix, graycoprops


processed_folder = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_
all_features = []


image_files = glob.glob(os.path.join(processed_folder, "*_processed.jpg"))

for img_path in image_files:
    filename_with_ext = os.path.basename(img_path)
    filename_without_ext = os.path.splitext(filename_with_ext)[0]
    category = filename_without_ext.split('_')[0]


    img = cv2.imread(img_path)
    if img is None:
        print(f"Warning: Could not load image {img_path}. Skipping.")
        continue


    mean_intensity = img.mean(axis=(0, 1))
    norm_intensity = np.linalg.norm(mean_intensity)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    glcm = graycomatrix(gray, distances=[1], angles=[0], levels=256, symmetric=True, normed=True)
    texture_features = {
        "contrast": float(graycoprops(glcm, 'contrast')[0, 0]),
        "homogeneity": float(graycoprops(glcm, 'homogeneity')[0, 0]),
        "energy": float(graycoprops(glcm, 'energy')[0, 0])
    }

    gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(gray_blur, 50, 150)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    shape_features = []
```

```python
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area < 100:
        continue

    x, y, w, h = cv2.boundingRect(cnt)
    if w < 10 or h < 10:
        continue

    perimeter = cv2.arcLength(cnt, True)
    M = cv2.moments(cnt)
    cx = int(M["m10"]/M["m00"]) if M["m00"] != 0 else 0
    cy = int(M["m01"]/M["m00"]) if M["m00"] != 0 else 0

    shape_features.append({
        "area": float(area),
        "perimeter": float(perimeter),
        "centroid": [cx, cy],
        "bounding_box": {"x": x, "y": y, "width": w, "height": h}
    })


shape_features = sorted(shape_features, key=lambda x: x["area"], reverse=True)[:5]


if not shape_features:
    shape_features = [{
        "area": 0.0,
        "perimeter": 0.0,
        "centroid": [0, 0],
        "bounding_box": {"x": 0, "y": 0, "width": 0, "height": 0}
    }]
```

Here we create a dictionary for a single image and appending image features to a list containing all the images and their feature extractions.

```
    image_data = {
        "filename": filename_with_ext,
        "category": category,
        "mean_intensity": mean_intensity.tolist(),
        "norm_intensity": float(norm_intensity),
        "texture_features": texture_features,
        "shape_features": shape_features
    }

    all_features.append(image_data)


with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/5DATA005W_W2120563_Dulwan_Senarath_image_feature_e
 json.dump(all_features, f, indent=4)

print("Feature extraction completed successfully!")
print("Saved as image_feature_extractions.json")
```

```
Feature extraction completed successfully!
Saved as image_feature_extractions.json
```

```
with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/5DATA005W_W2120563_Dulwan_Senarath_image_feature_e
    data = json.load(f)


for item in data[10:15]:
    print(json.dumps(item, indent=4))
    print("-" * 80)
```

Example Output:


```
"filename": "photography_image_20_processed.jpg",
"category": "photography",
"mean_intensity": [
    107.873312,
    103.626756,
    95.468676
],
"norm_intensity": 177.45259676354658,
"texture_features": {
    "contrast": 76.21070941883767,
    "homogeneity": 0.2599742482659511,
    "energy": 0.017374434910808634
},
"shape_features": [
    {
        "area": 2207.5,
        "perimeter": 180.75230813026428,
        "centroid": [
            27,
            191
        ],
        "bounding_box": {
            "x": 3,
            "y": 162,
            "width": 49,
            "height": 61
        }
    },
    {
        "area": 504.5
```

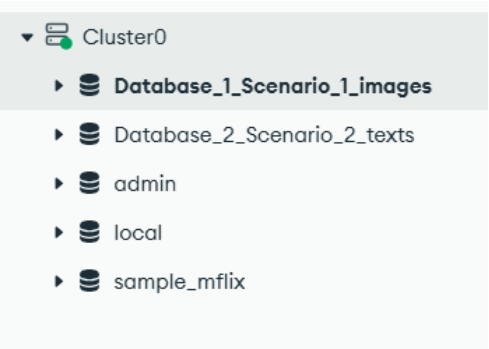
Database implementation and storage:

The database type used for this study is NoSQL – Document oriented one on MongoDB database platform because it allows for a schema to be designed flexibility, and also can efficiently handle heterogenous data types. Separate collections were created for **image annotation data**, **feature extraction data** and **processed images** assuming that this separation results in better modularity, scalability and query performance.

Collection name	Properties	Storage size	Documents
annotation metadata	-	53.25 kB	1
feature extractions	-	69.63 kB	75
processed images	-	6.05 MB	75

The processed images were persisted as binary data, and the metadata and features as well-structured JSON documents. This framework allows one to form complex queries which are based in a combination of visual features and semantic descriptions; for example image retrieval by category (semantic), texture(visual) properties and/or annotated keywords.

For the scenario 1, Database_Scenario_1_images database in Cluster0 is used.



First, the pymongo library is installed and necessary libraries are called.

Then the connection string obtained form MongoDB is pasted into the code and the password for the database also pasted.

After connected, the connection is tested by listing the collections of the relevant database.

```
!pip install pymongo
```

```
Collecting pymongo
  Downloading pymongo-4.15.5-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (22 k
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading pymongo-4.15.5-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (1.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/1.7 MB 22.0 MB/s eta 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 331.1/331.1 kB 19.7 MB/s eta 0:00:00
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.15.5
```

```
import pymongo
from pymongo import MongoClient
```

```
image_connections=pymongo.MongoClient("mongodb+srv://w2120563_DE_Database:DataEngineeringDatbase@cluster0.4vojjup.mongodb.ne
db_1=image_connections["Database_1_Scenario_1_images"]
```

```
try:
  print(db_1.list_collection_names())
except Exception as e:
  print(f"connection failed:{e}")
```

```
['feature extractions', 'annotation metadata', 'processed images']
```

After selecting the collection relevant collection name, the annotation metadata JSON file we created is uploaded to the database.

Inserting image annotation metadata to the database

```
image_annotation_metadata=db_1["annotation metadata"]
```

```
import json


with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/5DATA005W_W2120563_Dulwan_Senarath_annotation_metac
  image_annotations_data=json.load(file)

if isinstance(image_annotations_data, list):
  image_annotation_metadata.insert_many(image_annotations_data)
else:
  image_annotation_metadata.insert_one(image_annotations_data)
```

Same steps done for the “feature extractions” JSON file.

inserting image feature extraction data to the database

```
image_feature_extraction=db_1["feature extractions"]
```

```
with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/5DATA005W_W2120563_Dulwan_Senarath_image_feature_e
  image_feature_extractions=json.load(file)

if isinstance(image_feature_extractions, list):
  image_feature_extraction.insert_many(image_feature_extractions)
else:
  image_feature_extraction.insert_one(image_feature_extractions)
```

Finally, after importing necessary modules, we upload the processed images to the database.

inserting processed images to the database

```
processed_images=db_1["processed images"]
```

```
from bson import binary
processed_images_path="/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_Processed_Data_Engineering_Phc

for filename in os.listdir(processed_images_path):
  if filename.endswith("_processed.jpg"):
    with open(os.path.join(processed_images_path, filename), "rb") as f:
      binary_image=binary.Binary(f.read())
      image_doc={
          "filename":filename,
          "image":binary_image
      }
      processed_images.insert_one(image_doc)
```

Querying:

1. For processed images.

- To check whether the processed images are inserted to the database properly, first we check the count of the files in the processed images collection.

quering the database to see if files are loaded

for processed images

```
total_images = processed_images.count_documents({})
print(f"Total number of images: {total_images}")
Total number of images: 75
```

- The code returns that we have 75 images in total. So, we can assume that all the processed 75 images are inserted to the database.

- Next, picking one random image belonging to a category at a time, we check whether these categories exist in the database, just for extra clarification.

```
query_1={"filename":"food_image_12_processed.jpg"}
result=processed_images.find_one(query_1)
if result:
  print(f"found image:{result['filename']}")
else:
  print("image not found")
found image:food_image_12_processed.jpg
```

```
query_2={"filename":"product_image_12_processed.jpg"}
result=processed_images.find_one(query_2)
if result:
  print(f"found image:{result['filename']}")
else:
  print("image not found")
found image:product_image_12_processed.jpg
```

```
query_3={"filename":"dubai_image_7_processed.jpg"}
result=processed_images.find_one(query_3)
if result:
  print(f"found image:{result['filename']}")
else:
  print("image not found")
found image:dubai_image_7_processed.jpg
```

+ Code + Text

```
query_4={"filename":"photography_image_12_processed.jpg"}
result=processed_images.find_one(query_4)
if result:
  print(f"found image:{result['filename']}")
else:
  print("image not found")
found image:photography_image_12_processed.jpg
```

```
query_5={"filename":"transportation_image_12_processed.jpg"}
result=processed_images.find_one(query_5)
if result:
  print(f"found image:{result['filename']}")
else:
  print("image not found")
```

```
found image:transportation_image_12_processed.jpg
```

2. For feature extractions.

- Checking multiple feature records exists (sanity check)

```
results_fe_1 = image_feature_extraction.find().limit(5)

for doc in results_fe_1:
    print(doc["filename"], doc["texture_features"])
```

```
photography_image_8_processed.jpg {'contrast': 5.614613226452907, 'homogeneity': 0.6873313800948031, 'energy': 0.07819286258053944}
photography_image_12_processed.jpg {'contrast': 29.52092585170341, 'homogeneity': 0.40128152560097896, 'energy': 0.027088765895589135}
photography_image_11_processed.jpg {'contrast': 15.266733466933868, 'homogeneity': 0.43576229710890185, 'energy': 0.034525527369068366}
photography_image_19_processed.jpg {'contrast': 27.64400801603206, 'homogeneity': 0.5888506294001342, 'energy': 0.08754616993423033}
photography_image_13_processed.jpg {'contrast': 17.860336673346698, 'homogeneity': 0.3730958898694475, 'energy': 0.025253683516081316}
```

- Filtering images based on a feature condition.

```
results_fe_2 = image_feature_extraction.find(
    {"texture_features.contrast": {"$gt": 50}}
).limit(5)

for doc in results_fe_2:
    print(doc["filename"], doc["texture_features"]["contrast"])
```

```
transportation_image_5_processed.jpg 59.088833667334676
photography_image_16_processed.jpg 50.606072144288575
photography_image_20_processed.jpg 76.21070941883767
photography_image_5_processed.jpg 51.141891783567125
photography_image_7_processed.jpg 67.02214829659317
```

3. For annotation metadata.

- Retrieves multiple annotation entries and show that keywords and descriptions exist.

```
results_am_1 = image_annotation_metadata.find().limit(5)

for doc in results_am_1:
    print(doc)
```

```
{'_id': ObjectId('695d20e2adc728c56c8ae225'), 'dubai_image_1_processed': {'keywords': ['dubai skyline', 'palm island', 'luxury villas', 'blue la
```

- Confirm annotation metadata is loaded.

```
result_am_2 = image_annotation_metadata.find_one()

print("Number of annotated images:", len(result_am_2.keys()) - 1)
print("Sample keys:", list(result_am_2.keys())[1:6])
```

```
Number of annotated images: 75
Sample keys: ['dubai_image_1_processed', 'dubai_image_2_processed', 'dubai_image_3_processed', 'dubai_image_4_processed', 'dubai_image_5_processed
```

Scenario 2: Database for sentiment analysis models.

Methodology

Text collecting and sourcing

The dataset was prepared by gathering total 50 customer reviews of the Mazda 3 car for the sentiment analysis. Every review contains an opinion from a customer concerning their ownership and overall user experience with the car. There was manual compilation of the reviews into one text file, with every review considered a separate document necessary for the sentiment analysis.

This dataset has been chosen due to the factor that customer reviews naturally consist of both positive, negative and neutral sentiment, suitable for evaluating text preprocessing, feature extraction, and sentiment classification techniques.

A separate for the scenario 2 was created named “w2120563_text_data” where the raw text file is placed (“5DATA005W_W2120563_Dulwan_Senarath_car_reviews_for_DE_assignment”)

Text preprocessing

Text preprocessing was carried out to preprocess and standardize reviews before feature extractions. There were only two preprocessing methods used for this part,as given in the requirements.

1. Text normalization (lowercasing):

All the reviews were converted to lowercase to ensure consistency and to prevent duplicate tokens caused by case differences. (e.g., “Car” vs “car”)

2. Removal of punctuations and stop words:

Punctuation marks were removed because they do not contribute to sentiment meaning in vector-bases text representations. Stop words (e.g., the, is, and) were removed to filter out

noise from the text so that only sentiment-bearing words such as “excellent”, “faulty”, and “reliable” remained.

These steps helped in enhancing the quality of the data, reducing its dimensions, and focusing on the semantic meaning that the reviews had.

Code:

part B

```
import json
import re
import string
import nltk
from nltk.corpus import stopwords


nltk.download('stopwords')

stop_words = set(stopwords.words('english'))


def preprocess_text(text):

    text = text.lower()


    text = text.translate(str.maketrans('', '', string.punctuation))


    text = re.sub(r'\d+', '', text)


    tokens = text.split()
    tokens = [word for word in tokens if word not in stop_words]

    return " ".join(tokens)


file_path = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_text_data/5DATA005W_W212

with open(file_path, "r", encoding="utf-8") as file:
    raw_text = file.read()
```

```
reviews = re.split(r'\n\d+,', raw_text)
reviews = [review.strip() for review in reviews if review.strip()]


cleaned_reviews = [preprocess_text(review) for review in reviews]


for i in range(3):
    print(f"Review {i+1} (cleaned):\n{cleaned_reviews[i]}\n")
```

Example output:

```
Review 1 (cleaned):
car excellent even though isnt performance car slightest take spirited drives occasional track days genuinely

Review 2 (cleaned):
absolutely beautiful cars bp series sedan hatch mine dec hope enjoy till retire driving also w w ba falcon xt

Review 3 (cleaned):
excellent car price – ive really impressed bl mazda 1 petrol broken three years average fuel economy around 1

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Feature extraction / vectorization

Two common techniques of feature extraction, efficient in computation were used,

1. Bag of Words (BoW)

BoW was used as an approach that translated each review into a vector based on words frequency within the documents, where the vocabulary is analyzed and each review becomes a row and each unique word Becomes a column.

2. TF-IDF (Term Frequency-Inverse Document Frequency)

TD-IDF was used to minimize the effects of frequently occurring words and assign higher importance to words with more discriminative capabilities among reviews. By using both approaches, one can compare simple frequency-driven representation with use of term improtance.

Using both methods allows comparison between simple frequency-based representation and higher term importance.

Code:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

# -------- Bag of Words --------
bow_vectorizer = CountVectorizer()
bow_matrix = bow_vectorizer.fit_transform(cleaned_reviews)

# -------- TF-IDF --------
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(cleaned_reviews)

# check shapes
print("Bag of Words shape:", bow_matrix.shape)
print("TF-IDF shape:", tfidf_matrix.shape)
```

Output:

```
Bag of Words shape: (50, 1284)
TF-IDF shape: (50, 1284)
```

Sentiment labeling

The labels for the sentiment analysis (positive, negative, neutral) were all auto coded using VADER sentiment analysis tool. This helped to ensure that all reviews identified as cleaned were given a polarity score for their compound, and then all corresponding labels were determined by VADER thresholds using a for loop.

This helped to ensure that all reviews are accurately coded. If the compound score is equal or greater than 0.05 (clearly positive), it is considered as positive. If the score is equal or less than -0.05(clearly negative), it is considered negative and if the score is not clearly positive or negative (between 0.05 and -0.05, it is considered neutral.

Code:

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
nltk.download('vader_lexicon')

sia = SentimentIntensityAnalyzer()
sentiment_labels = []

for review in cleaned_reviews:
    score = sia.polarity_scores(review)
    compound = score['compound']

    if compound >= 0.05:
        sentiment_labels.append("positive")
    elif compound <= -0.05:
        sentiment_labels.append("negative")
    else:
        sentiment_labels.append("neutral")


for i in range(5):
    print(sentiment_labels[i])
```

Output:

```
positive
positive
positive
positive
positive
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
```

Creating files to save in the database

- Processed reviews with sentiments (text file)

Here we have made a text file out of the review, along with its review id and sentiments.

```
output_directory = "/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_text_data"
output_filename = "5DATA005W_W2120563_Dulwan_Senarath_processed_reviews_with_sentiment.txt"
output_file_path = os.path.join(output_directory, output_filename)

with open(output_file_path, "w", encoding="utf-8") as f:
    for i, review in enumerate(cleaned_reviews):
        f.write(f"Review {i+1}:\n")
        f.write(f"Text: {review}\n")
        f.write(f"Sentiment: {sentiment_labels[i]}\n")
        f.write("\n" + "-"*50 + "\n\n")

print("Processed reviews with sentiment saved as .txt successfully!")
```

```
Processed reviews with sentiment saved as .txt successfully!
```

- Metadata file (JSON)

```
reviews_metadata_list = []
for i, review_text in enumerate(cleaned_reviews):
    reviews_metadata_list.append({
        "review_id": i + 1,
        "sentiment_label": sentiment_labels[i]
    })

metadata_2 = {
    "dataset_description": "This dataset contains car reviews used for sentiment analysis and text vectorisa
    "text_preprocessing": {
        "normalisation": "All text was converted to lowercase to ensure consistency.",
        "punctuation_removal": "All punctuation marks were removed from the text.",
        "stopword_removal": "Common English stop words were removed using the NLTK stopword list.",
        "number_removal": "Numeric values were removed as they do not contribute to sentiment meaning."
    },
    "vectorisation_methods": {
        "bag_of_words": {
            "description": "Bag of Words represents text by counting the frequency of each word in a review.
            "tool_used": "sklearn.feature_extraction.text.CountVectorizer",
            "output": "Sparse matrix where rows represent reviews and columns represent word frequencies."
        },
        "tf_idf": {
            "description": "TF-IDF assigns weights to words based on their importance in a review relative t
            "tool_used": "sklearn.feature_extraction.text.TfidfVectorizer",
            "output": "Sparse matrix with weighted word importance values."
        }
    },
    "sentiment_labelling": {
        "method": "VADER Sentiment Intensity Analyzer",
        "labels": {
            "positive": "Compound score \u2265 0.05",
            "neutral": "Compound score between -0.05 and 0.05",
            "negative": "Compound score \u2264 -0.05"
        }
    },
    "reviews_data": reviews_metadata_list
}

with open("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_text_data/5DATA005W_W21205
    json.dump(metadata_2, f, indent=4)
```

Database implementation and storage

part e : database design

```
!pip install pymongo
```

```
Requirement already satisfied: pymongo in /usr/local/lib/python3.12/dist-packages (4.15.5)
Requirement already satisfied: dnspython<3.0.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from pymongo) (2.8.
```

```
import pymongo
from pymongo import MongoClient
```

```
text_connections=pymongo.MongoClient("mongodb+srv://w2120563_DE_Database:DataEngineeringDatbase@cluster0.4vojjup.mongo
db_2=text_connections["Database_2_Scenario_2_texts"]
```

```
try:
  print(db_2.list_collection_names())
except Exception as e:
  print(f"connection failed:{e}")
```

```
['processed reviews with sentiments', 'metadata (JSON)']
```

Inserting metadata (JSON) into the database.

inserting metadata (JSON) to the database

```
text_json_metadata=db_2["metadata (JSON)"]
```

```
with open ("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_text_data/5DATA005W_W2120563_Dulwan
  text_json_metadata_data=json.load(file)

if isinstance(text_json_metadata_data, list):
  text_json_metadata.insert_many(text_json_metadata_data)
else:
  text_json_metadata.insert_one(text_json_metadata_data)
```

Inserting processed reviews with sentiments to the database.

inserting processed reviews with sentiments to database

```
processed_reviews_with_sentiments=db_2["processed reviews with sentiments"]
```

```
with open ("/content/drive/MyDrive/W2120563_Data_Engineering_Assessment_2/w2120563_text_data/5DATA005W_W2120563_Dulwan

  processed_reviews_with_sentiments_data = file.read()

processed_reviews_with_sentiments.insert_one({"full_text_reviews": processed_reviews_with_sentiments_data})
```

```
InsertOneResult(ObjectId('695dab7cf50788efb54c78dc'), acknowledged=True)
```

Querying:

Two simple queries are performed to check whether databases were implemented correctly.

```
for doc in processed_reviews_with_sentiments.find().limit(1):
    print(doc)
```

```
ics compared old bl model drove like go kart guess take time get used certainly comfortable overly aggressive radar crui
```

```
metadata_document = text_json_metadata.find_one(
    {},
    {"_id": 0, "dataset_description": 1}
)

if metadata_document:
    print(json.dumps(metadata_document, indent=4))
else:
    print("No metadata document found in 'text_json_metadata' collection.")
```

```
{
    "dataset_description": "This dataset contains car reviews used for sentiment analysis and text vectorisation."
}
```

Evaluation

In the coursework, the design and implementation if two separated NoSQL databases for different types of data, including images for the CBIR system and texts for sentiment analysis, have been examined. In both systems MongoDB platform has been chosen as the NoSQL database since its document-based model for flexible storage facilities regarding images, JSON files and text files.

In the case of scenario 1, the preprocessing techniques made the image dataset homogenous by resizing. The attachment of well-structured annotation data (image keywords and descriptions) helps with semantic analysis and future retrievals tasks. Also extracting multiple feature types increase representation quality of the image database. However, some limitations still remain, such as dependance of manual designed features and image metadata.

For scenario 2, pre-processing steps such as normalizing text data, removing punctuation, and removing stop-words were necessary to improve the quality of the data before converting to vectors. Application of the Bag of Words algorithm and TD-IDF created meaningful numeric vector ideal for sentiment analysis. Though basic text classification to determine sentiment has readability and understandability in lexicon-based models, they possibly lack context-specific accuracy in sarcasm presented in some reviews.

However, this developed workflow meets all the requirements needed in the assessment and shows a deep understanding of text processing and NoSQL database design.

Conclusion

This coursework has successfully fulfilled its objectives in designing and developing two NoSQL databases based on distinct types of data. In scenario 1, a Content-Based Image Retrieval database has been designed using images dataset with variation in its content. This MongoDB database has been successfully developed by storing processed images and features that can support efficient retrievals.

The sentiment database has been developed for customer review text data which underwent text preprocessing and vectorization, understanding reviews in a sentiment-wise manner, translating meaningless text data into insightful numeric representations that have been stored alongside metadata in MongoDB.

References

Images dataset:

***Note**: Random images have been used in this given links. The raw images are shared via google drive folder.*

Dubai images - https://www.pexels.com/search/dubai/

Product images - https://www.pexels.com/search/products/

Transportation images - https://www.pexels.com/search/transportation/

Food images - https://www.pexels.com/search/food/

Photography images - https://www.pexels.com/search/photography/

Text dataset:

***Note**: Random customer reviews are fetched from the below website. The text file containing the reviews are shared via google drive folder.*

Car reviews - https://www.productreview.com.au/listings/mazda-3

Main google drive link =

https://drive.google.com/drive/folders/1QR5gVCRAZA48dr5q9Ldgx9U8UnJ3cQgy?usp=sharing

username of the database = w2120563_DE_Database

password = DataEngineeringDatbase

databse link =